

TECHNICAL REPORT · 2026

verl 大模型强化学习训练框架： 从 0 到 1 的系统掌控

HybridFlow 编程模型、Ray 单控制器、DataProto 数据协议、Worker/Engine 分层、PPO/GRPO/DAPO 映射、性能调优与扩展路线的完整说明。

Winston

V1.0 · 2026.05.18

Sources checked against verl official repository, official docs and primary papers

| 目录

01	执行摘要：先建立一张完整地图	03
02	问题定义：LLM 强化学习后训练在解决什么	05
03	架构总览：HybridFlow 的核心判断	07
04	控制平面：Ray、ResourcePool、WorkerGroup	09
05	数据协议：DataProto 如何让分布式调用像单机函数	11
06	训练链路：从 parquet 到 update_actor	13
07	Worker 与 Engine：角色、后端与权重同步	15
08	算法层：PPO、GRPO、DAPO 与优势估计	17
09	配置层：Hydra 参数如何改变训练行为	19
10	工程落地：性能、调试、异步、Agentic RL	21
11	附录：源码路线、术语表与来源	23

执行摘要：先建立一张完整地图

verl 是面向大语言模型后训练的强化学习训练框架。它真正要解决的不是「如何写一个 ppo loss」，而是如何把 **多模型、多阶段、多后端、多 GPU** 的 RLHF / RLVR 训练组织成一条可扩展、可替换、可调试的数据流。

本文口径

本文资料核对于 2026-05-18。官方仓库 `main` 浅克隆 HEAD 为 657cfa5，版本文件为 0.8.0.dev；远端标签列表中最新标签为 v0.7.1。因此本文把「主线开发能力」和「稳定发布标签」分开表述。

一句话理解 verl

在普通监督训练中，训练循环通常是数据进来、模型前向、loss、反向、优化器更新。LLM 强化学习后训练多出至少 5 个环节：当前策略生成回答、旧策略或 rollout 策略计算 log probability、参考策略计算 KL、奖励函数或奖励模型打分、critic 或 group baseline 估计 advantage。每个环节可能使用不同模型、不同并行方式、不同 GPU 放置和不同推理引擎。verl 的主张是：**算法控制流由单控制器写成清晰的 Python 程序，重计算由分布式 Worker 和 Engine 执行。**

核心结论

- **HybridFlow 是架构主轴。**它把 RL 算法看成高层 dataflow: rollout、log-prob、reward、advantage、update_actor、update_critic 是高层算子；FSDP、Megatron、vLLM、SGLang 内部的 tensor 并行和 collective 通信是底层 computation flow。
- **Ray 单控制器降低算法开发成本。**开发者在 driver 进程里像写单机程序一样调用 `actor_rollout_wg.generate_sequences(batch)`，WorkerGroup 负责把数据切给远端 worker、触发 RPC、收集结果。
- **DataProto 是 verl 的数据契约。**它把 tensor batch、非 tensor 元数据和 meta 信息放在同一容器里，支持 pop、union、chunk、concat 等操作，让高层训练循环不需要知道每个后端的 rank 拓扑。
- **Worker 与 Engine 分层是可扩展性的关键。**ActorRolloutRefWorker 负责 actor、rollout、reference policy 的组合；TrainingWorker 包住具体 BaseEngine；EngineRegistry 决定使用 FSDP、FSDP2、Megatron、Automodel、VeOmni 或 TorchTitan 等后端。
- **配置不是附属品，而是训练语义。**`data.train_batch_size`、`rollout.n`、`ppo_mini_batch_size`、`ppo_micro_batch_size_per_gpu`、`adv_estimator`、`use_kl_loss`、`use_kl_in_reward` 共同决定算法行为、吞吐、显存和收敛稳定性。

读者应该掌握的 5 层模型

层级	你要回答的问题	verl 中的对象	掌握标准
算法层	我到底在优化什么目标？	PPO, GRPO, DAPO, RLOO, REINFORCE++	能解释 reward、advantage、KL、clip ratio 的作用。
控制层	每一步先后顺序是什么？	RayPPOTrainer, main_ppo_sync, TaskRunner	能手画 rollout -> reward -> advantage -> update 的循环。
数据层	各阶段传递什么字段？	DataProto, TensorDict, non_tensor_batch	能说清 prompt、responses、old_log_probs、values、token_level_rewards 的来源。
执行层	哪个模型在哪些 GPU 上跑？	ResourcePool, WorkerGroup, ActorRolloutRefWorker	能设计 colocate 或 disaggregate 放置方案。
后端层	训练与生成由谁实际计算？	BaseEngine, vLLM, SGLang, CheckpointEngine	能根据模型规模选择 FSDP2、Megatron 和 rollout TP。

学会 verl 的标志不是记住某个脚本，而是看到一组配置和日志后，能定位「这一步是在 controller 上算，还是在 worker 上算；数据是不是 DataProto；瓶颈是 rollout、log-prob、critic、reward 还是权重同步」。

问题定义：LLM 强化学习后训练在解决什么

大模型强化学习后训练把语言模型视为策略。prompt 是状态的一部分，生成 token 是动作，完整回答是轨迹，奖励来自人类偏好模型、规则验证器、沙箱测试、环境反馈或它们的组合。

为什么 SFT 不够

监督微调 SFT 让模型模仿给定答案，优化的是「目标 token 的似然」。这适合灌输格式、风格和基础任务能力，但它不能直接表达「多个回答都像训练集，但只有一个通过单元测试」「推理链很长，中间没有逐 token 标签」「工具调用需要和环境互动」这类目标。RLVR 和 RLHF 把评价函数放到生成之后：模型先完整生成，再由 reward 给出分数。

因此后训练任务从一个普通 batch training 问题变成一个闭环：生成会消耗大量推理算力，奖励可能在 CPU、GPU、远程沙箱或外部环境里执行，训练又要把生成样本变成可求梯度的 policy loss。verl 的工程价值就在这里：它把闭环拆成明确阶段，并让每个阶段能放到最合适的后端。

LLM RL 的基本变量

符号 / 字段	直观含义	在 verl 中常见位置	错误会造成什么
q / prompt	题目、对话历史或环境观测	<code>data.prompt_key</code> , <code>parquet prompt</code>	chat template 不一致、长度截断、reward 找不到 ground truth。
o / response	策略生成的回答 token 序列	<code>responses</code> , <code>response_mask</code>	EOS mask 错误会污染 loss 和 reward。
R	序列级或 token 级奖励	<code>token_level_scores</code> , <code>token_level_rewards</code>	奖励尺度过大导致 KL 爆炸，过稀导致方差高。
A	advantage, 回答相对基线的好坏	<code>advantages</code> , <code>returns</code>	估计器不匹配会使 PPO/GRPO 行为偏离预期。
$\log \pi_{\theta}$	当前 actor 对生成 token 的 log probability	<code>old_log_probs</code> , <code>log_probs</code>	训练/生成不一致会导致 ratio 失真。
$\log \pi_{\text{ref}}$	参考策略 log probability	<code>ref_log_prob</code>	KL 约束失效，模型容易 reward hacking。

从 RLHF 到 RLVR 和 Agentic RL

RLHF 传统上依赖奖励模型或偏好模型给回答打分。RLVR 更常见于数学、代码、形式化证明和检索任务：奖励来自可验证规则，例如最终答案匹配、测试用例通过、定理证明检查器接受。Agentic RL 再进一步，回答不再是单轮文本，而是多轮 action：模型调用搜索、代码解释器、浏览器、数据库或外部环境，reward 可能在多次交互后才出现。

这三类任务要求同一个框架同时支持多种 reward 来源、长序列、多轮轨迹、异步生成、外部工具调用和训练后端切换。verl 官方文档把这些能力分散在 preparation、reward_function、sglang_multiturn、agent_loop、fully_async 等章节中；本文把它们收束到一条主线：数据怎么进来，模型怎么生成，奖励怎么进入 batch，梯度怎么回到 actor。

一个最小闭环

1. 把原始数据预处理成 parquet，至少包含 prompt，通常还包含 data_source、reward_model.ground_truth、extra_info。
2. 用当前 actor 作为 rollout policy，针对每个 prompt 生成 n 条回答。
3. 计算旧策略 log probability 和参考策略 log probability，用于 PPO ratio 或 KL 约束。
4. 调用 reward function、reward model 或环境，得到序列级或 token 级分数。
5. 根据 PPO、GRPO、DAPO 等算法计算 advantage 和 returns。
6. 更新 actor，必要时更新 critic，然后把新权重同步给 rollout engine。
7. 按 test_freq 验证，按 save_freq 保存 checkpoint，持续记录 rollout、reward、KL、长度和吞吐指标。

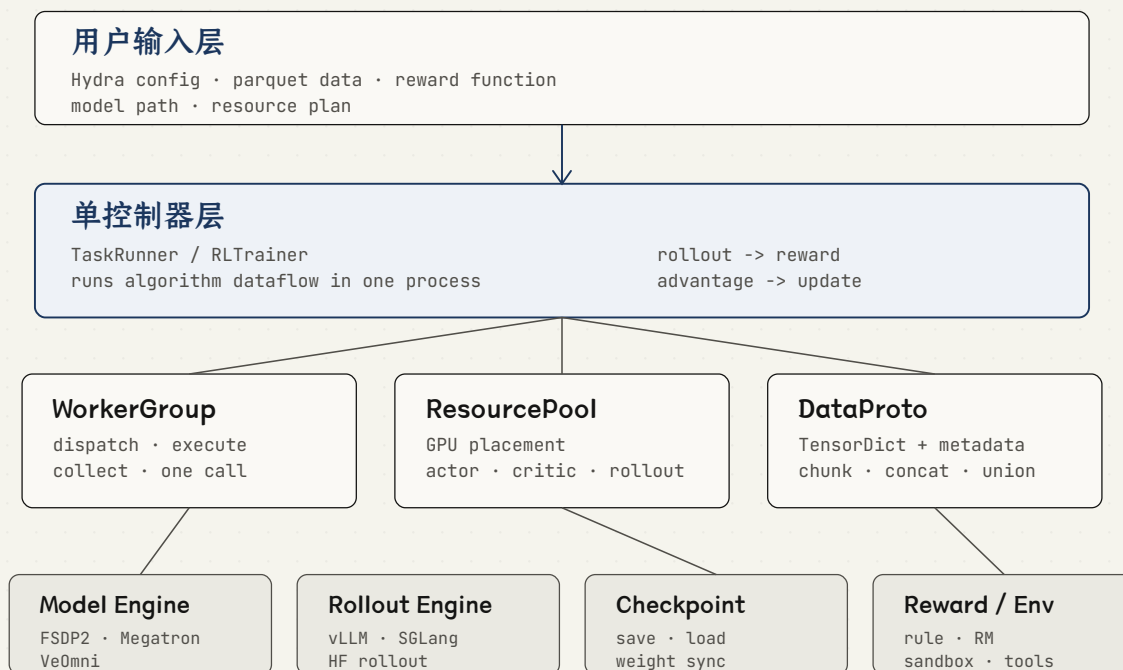
从 0 到 1 的第一条判断

不要先问「我该用 PPO 还是 GRPO」。先问「我的 reward 是谁给的，reward 是否稀疏，是否需要 critic，是否每个 prompt 要采多条回答，生成是否是单轮，训练和生成是否能共用 GPU」。这些答案会自然指向算法和工程配置。

架构总览：HybridFlow 的核心判断

HybridFlow 的关键判断是把 RL 训练拆成两层：高层控制流由一个 driver 进程表达，底层计算流由多进程训练和推理后端执行。这个折中牺牲了一部分 driver 与 worker 之间的数据通信开销，换来算法可读性、后端可替换性和资源放置灵活性。

FIGURE 1 VERL HYBRIDFLOW MENTAL MODEL



HybridFlow 的重点不是某个后端，而是把算法控制、资源放置、数据协议和底层计算分开。

为什么不用纯多控制器

在传统分布式训练中，常见做法是每个 rank 跑同一段程序，DDP 或 FSDP 负责同步。这个模式适合单模型训练，但 RLHF 有多个模型和多个 DAG：actor 负责策略更新，rollout 负责生成，reference 负责 KL，critic 负责 value，reward 可能是另一个模型或外部函数。若把控制流也写成多进程程序，算法逻辑会和某个后端的并行细节强绑定，换 FSDP 到 Megatron 或调整 actor/rollout 放置都会牵动控制代码。

verl 选择把高层控制流留在一个单进程 driver 中。这样写 PPO 或 GRPO 的人可以直接表达「先生成、再打分、再更新」；FSDP、Megatron、vLLM、SGLang 的复杂分布式实现隐藏在 Worker 和 Engine 里面。代价是 controller 与 worker 之间存在数据往返，但换来的是算法可写、后端可换、资源可调。

两层 dataflow

层	节点是什么	边上传什么	谁负责
RL 控制流	rollout、log-prob、reward、advantage、actor update	DataProto / TensorDict / BatchMeta	RLTrainer / RayPPOTrainer / main_ppo_sync
模型计算流	forward、backward、optimizer step、KV cache、weight sync	tensor shard、parameter、gradient、activation	FSDP2、Megatron、vLLM、SGLang、CheckpointEngine

verl-core 与 verl-trainer

官方 v0.7 release blog 把主线架构拆成两块：verl-core 提供 Model Engine、Rollout Engine、Checkpoint Engine、TransferQueue 等可插拔基础组件；verl-trainer 在这些组件之上组合 on-policy、one-step-off-policy 和 fully async 等训练 pipeline。这个拆分很重要，因为它说明 verl 不是只有一个 PPO trainer，而是在向「可组合 RL 训练系统」演进。

对使用者而言，最保守的路线是从同步 on-policy pipeline 开始：actor 与 rollout 通常 colocate，算法最接近标准定义，调试最直接。等瓶颈明确后，再考虑一步 off-policy 或全异步，把生成和训练重叠，或把 rollout server 从训练节点中拆出去。

HybridFlow 的教授级解释可以压缩为一句话：它把「算法的顺序」和「算子的并行实现」解耦。前者追求可读，后者追求吞吐；二者通过 DataProto、WorkerGroup 和 Engine API 接上。

控制平面：Ray、ResourcePool、WorkerGroup

verl 的控制平面把分布式系统包装成一组可调用对象。你在 driver 上调用一个方法，背后发生的是数据切分、Ray actor 远程执行、结果收集和重新拼接。

入口：Hydra main 到 TaskRunner

典型 PPO 入口在 verl/trainer/main_ppo.py，当前源码已标注该文件将被 main_ppo_sync.py 替代。入口函数由 Hydra 加载 ppo_trainer 配置，然后初始化 Ray。真正的训练主体不是直接在当前进程里跑，而是创建一个 Ray remote TaskRunner，再在它的 run(config) 中完成 worker 映射、数据集构造、trainer 初始化和 fit()。

```
main(config)
  → auto_set_device(config)
  → migrate_legacy_reward_impl(config)
  → run_ppo(config)
    → ray.init(runtime_env)
    → TaskRunner.remote()
    → runner.run.remote(config)
```

这个设计有两个实际意义。第一，driver 可以不跑在 Ray head 节点上，避免 head 节点资源不足导致内存压力。第二，所有资源、worker 和 trainer 对象都在 Ray 管理范围内创建，便于跨节点调度。

ResourcePool: GPU 资源怎么分组

ResourcePool 或 RayResourcePool 把集群 GPU 划成逻辑池。最简单配置是一个 global_pool，actor、rollout、critic、reference、reward 都映射到同一组 GPU。这种 colocate 模式更容易起步，也方便 actor 和 rollout 之间做快速权重同步。

当任务变大时，资源池会变成策略问题：reward model 是否单独占 GPU，teacher model 是否单独占 GPU，rollout 是否需要更多推理副本，critic 是否可以和 actor 分开，Megatron 的不同角色是否需要不同 3D 并行配置。verl 的 mapping 设计让这些策略不必侵入训练循环。

资源放置	适合场景	优点	风险
全 colocate	单机或小集群起步、严格 on-policy baseline	配置简单，权重同步短路径，调试容易	rollout 和训练互相挤显存，长尾生成会让训练等待。
reward 单独池	奖励模型较大或验证成本高	避免 reward 占用 actor/rollout 显存	reward 结果回传成为额外数据依赖。
rollout 单独池	长 CoT、多轮工具调用、生成瓶颈明显	推理可弹性扩展，训练和生成可重叠	权重同步和 stale policy 控制变复杂。
teacher 单独池	on-policy distillation 或大 teacher	teacher 查询不阻塞 actor 显存	需要监控 teacher wait time。

WorkerGroup: 一个方法背后的三段式

WorkerGroup 是用户最应该理解的控制平面对象。它管理多个远端 worker，但暴露成一个本地对象。verl 文档把一次分布式方法调用拆成 3 个动作：**dispatch**、**execute**、**collect**。

- 1. **dispatch**: 把输入 DataProto 按 worker world size 切分，或者把同一个配置复制给所有 worker。
- 2. **execute**: 通过 Ray RPC 触发每个 worker 上的同名方法。
- 3. **collect**: 把每个 worker 的返回值收集、拼接、聚合成 driver 继续能用的数据。

```
@register(dispatch_mode=Dispatch.DP_COMPUTE_PROTO)
def generate_sequences(self, prompts: DataProto):
    pass # worker-side generation

# driver 侧看起来像普通函数
output = actor_rollout_ref.wg.generate_sequences(batch)
```

dispatch mode 的意义

模式	做什么	典型用途	思考方式
ONE_TO_ALL	同一份参数发给所有 worker	init_model , save_checkpoint	每个 rank 都要做同一件事。
DP_COMPUTE_PROTO	按数据并行维度切 DataProto	rollout、log-prob、value、reward	每个 worker 处理 batch 的一片。
make_nd_compute_dataprote_dispatch_fn	根据 engine mesh 生成 n-d dispatch	Megatron PP/TP/DP 组合下的 log-prob 和 update	后端拓扑被映射成 controller 可理解的数据分发。

掌握标准

看到一个 worker 方法时，先看它的 @register。这比先看函数体更重要，因为它告诉你该方法是在所有 worker 同步执行，还是按数据切片执行，返回值是否会被拼回 driver。

数据协议：DataProto 如何让分布式调用像单机函数

DataProto 是 verl 的共同语言。没有它，rollout、reward、critic、actor update 之间会被各种后端的 tensor 格式、Python 元数据和 Ray object ref 拆散。

三类内容

部分	类型	放什么	例子
batch	TensorDict	可按 batch 维切分的 tensor	input_ids , attention_mask , responses , old_log_probs , advantages
non_tensor_batch	dict	每条样本对应的非 tensor 信息	data_source , reward_model , extra_info , uid
meta_info	dict	这次调用的全局控制信息	eos_token_id , pad_token_id , validate , global_steps

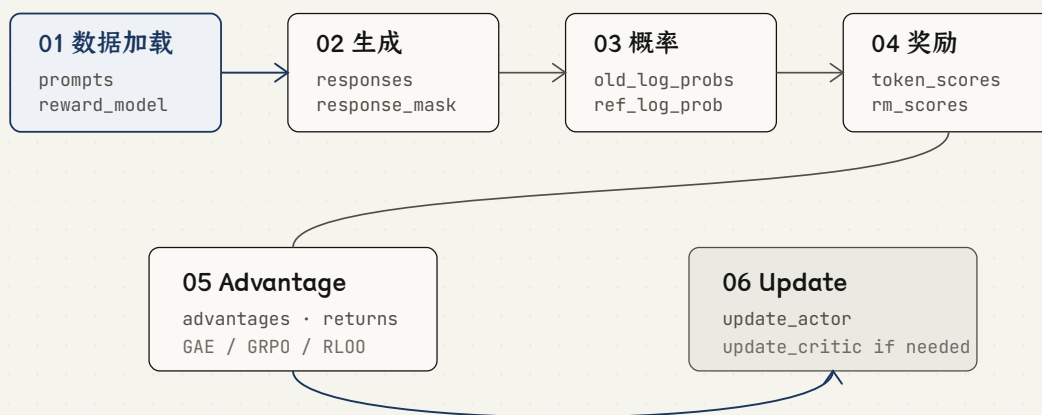
为什么不是普通 dict

RL 训练的 batch 会经历多次增删：生成前只需要 prompts、attention mask 和 position ids；生成后新增 responses；log-prob 后新增 old log probability；reward 后新增 token-level scores；advantage 后新增 advantages 和 returns。普通 dict 无法稳定表达「tensor 字段按 batch 切、非 tensor 字段按样本跟随、meta 信息原样复制」这三种行为。

DataProto 提供 pop、select、union、chunk、split、concat、repeat 等操作，正好覆盖训练循环需要的所有数据变形。它不是算法模块，但它决定了算法模块之间是否能组合。

一次 PPO batch 的字段演化

FIGURE 2 DATAPROTO FIELD EVOLUTION



DataProto 让每个阶段只追加或取出自己关心的字段，训练循环由字段演化连接起来。

DataProtoFuture 与异步趋势

当前主线源码中还有 DataProtoFuture，目标是避免 driver 过早 materialize 数据。它保存来自另一个 WorkerGroup 的 future 列表，并携带 collect 与 dispatch 函数；当下游 worker 真正需要数据时再取。这个对象体现了 verl 后续异步化方向：控制流继续在 driver 上表达，但数据流尽量不被 driver 拿回内存。

v0.7 release blog 进一步提出 TransferQueue，用 BatchMeta / KVBatchMeta 等元数据引用真实 tensor 数据，降低大规模多模态与异步训练中的数据搬运成本。本文把它视为主线演进方向，而不是所有稳定用户都必须立即采用的默认入口。

如果你的自定义模块不能接入 verl，先检查它是否遵守 DataProto 契约：batch 维是否一致，non_tensor_batch 是否逐样本对齐，meta_info 是否只放控制信息，返回值是否能被 WorkerGroup collect。

| 训练链路：从 parquet 到 update_actor

verl 的训练链路可以理解成一条状态机。每一步都消费一个 DataProto，产生一个带有更多字段的 DataProto，最后交给 actor 和 critic 更新。

第一步：准备 parquet 数据

官方快速开始以 GSM8K 为例，把 Hugging Face dataset 转成 parquet。最小字段是 `prompt`，但强化学习训练通常还需要 `data_source`、`ability`、`reward_model.ground_truth`、`extra_info`。这些字段不是装饰，它们决定 reward manager 如何选择打分函数。

```
{
  "data_source": "openai/gsm8k",
  "prompt": [{"role": "user", "content": "question text"}],
  "ability": "math",
  "reward_model": {"style": "rule", "ground_truth": "42"},
  "extra_info": {"split": "train", "index": 17}
}
```

自定义数据的第一条纪律是：reward function 使用的 ground truth 口径必须和数据预处理完全一致。数学任务常见问题是最终答案提取规则不同；代码任务常见问题是测试用例、输入输出格式和沙箱约定没有进入 `extra_info`。

第二步：创建 dataset 和 dataloader

`RLHFDataset` 读取 parquet，应用 chat template，处理 padding 与 truncation，并生成 `input_ids`、`attention_mask`、`position_ids` 等字段。`StatefulDataLoader` 支持训练中断恢复时保留采样状态。`data.train_batch_size` 是每次训练迭代抽取的 prompt 全局数量，GRPO/DAPO 中如果 `rollout.n > 1`，实际生成回答数量是 `train_batch_size * n`。

第三步：rollout 生成

trainer 会从 batch 中取出生成所需字段，构造 `gen_batch`，然后调用 `actor_rollout_wg.generate_sequences(gen_batch)`。这一步由 rollout backend 实际执行，常见后端是 vLLM 或 SGLang；调试或小规模探索可以使用 HF rollout。生成结果包含 `responses` 和相关 `mask`，然后与原 batch 合并。

生成阶段通常是 RL 训练最大的时间成本。长链推理、工具调用和多轮 Agent 会产生长尾请求，导致同步训练等待最慢样本。verl 的异步 rollout、server mode、AgentLoop 和 fully async pipeline 都是在缓解这个问题。

第四步：概率、奖励与优势

PPO 需要当前策略对生成 token 的旧 log probability，也需要 reference policy 的 log probability 来控制 KL，还需要 critic 给出 value。GRPO 不训练 critic，而是对同一 prompt 的多条回答做 group baseline。DAPO 在 GRPO 类思路上加入动态采样、clip higher、token-level loss 和 overlong reward shaping。

```

batch = batch.union(gen_batch_output)
batch = batch.union(actor_rollout_wg.compute_log_prob(batch))
batch = batch.union(ref_policy_wg.compute_ref_log_prob(batch)) # if KL needed
batch = batch.union(critic_wg.compute_values(batch))           # if critic needed
batch.batch["token_level_scores"] = reward_fn(batch)
batch = compute_advantage(batch, adv_estimator=config.algorithm.adv_estimator)

```

第五步：更新 actor 和 critic

当 batch 包含 responses、old log probabilities、rewards、advantages 等字段后，actor worker 就可以按 mini-batch 和 PPO epoch 训练。critic 若启用，也会更新 value head。更新完成后，metrics 被收集回 driver，用于 logger、验证和 checkpoint。

真正要理解的是 batch size 的三层含义：train_batch_size 是采多少 prompt，ppo_mini_batch_size 是一次优化看多少轨迹，ppo_micro_batch_size_per_gpu 或 dynamic token limit 是每张 GPU 一次 forward/backward 承受多少样本或 token。第一类影响算法，第三类主要影响显存和吞吐。

第六步：验证、保存与恢复

trainer.test_freq 控制验证频率，trainer.save_freq 控制 checkpoint 频率，trainer.resume_mode 控制从 checkpoint 自动恢复还是指定路径恢复。官方配置默认 checkpoint 存在 checkpoints/\${trainer.project_name}/\${trainer.experiment_name}。FSDP 与 Megatron 的中间 checkpoint 通常是分片格式，若要部署为 Hugging Face 模型，需要走对应 merge 工具。

训练循环记忆法

先生成，后定价，再归因，最后更新。生成给出行为，概率给出 ratio 和 KL，奖励给出方向，advantage 给出归因，actor update 才真正改变模型。

Worker 与 Engine：角色、后端与权重同步

verl 的 Worker 不是简单进程，而是「角色组合」。同一个 ActorRolloutRefWorker 可以只构造 actor，也可以构造 actor + rollout，也可以构造 actor + rollout + ref。真正的训练实现进一步交给 TrainingWorker 和 BaseEngine。

角色拆解

角色	做什么	常见对象	什么时候需要
Actor	被优化的策略模型	TrainingWorker + LM engine	所有 RL 后训练都需要。
Rollout	用当前或近似当前策略生成回答	BaseRollout + vLLM/SGLang/HF	on-policy、GRPO、Agentic RL 都需要。
Reference	提供 KL 约束基准	forward-only TrainingWorker	PPO KL reward 或 GRPO KL loss 需要。
Critic	估计 value，降低方差	value model TrainingWorker	PPO/GAE 需要，GRPO 通常不需要。
Reward	给回答打分	rule function、reward model、sandbox、env	所有 RL 都需要某种 reward。
Teacher	蒸馏或知识提供方	teacher pool / server	on-policy distillation 等场景。

ActorRolloutRefWorker

官方 Engine Workers 文档把 ActorRolloutRefWorker 定义为 hybrid worker。它的 role 字段决定 init_model 里构造哪些子对象：actor 构造 actor TrainingWorker 和 checkpoint engine；rollout 构造 rollout engine；ref 构造 forward-only reference TrainingWorker；actor_rollout 或 actor_rollout_ref 则组合这些角色。

这种组合有很强的工程目的：actor 与 rollout colocate 时，更新后的 actor 权重可以通过本地或 NCCL 路径更快送到 rollout engine；LoRA 场景中 reference policy 可以被视为不带 adapter 的 base model，减少额外模型常驻成本。

TrainingWorker 与 BaseEngine

TrainingWorker 是一个 Ray-wrapped engine API。它负责 reset、to device/offload、set_loss_fn、train_mini_batch、train_batch、infer_batch、save_checkpoint、load_checkpoint。它不关心底层是 FSDP2 还是 Megatron，底层实现由 EngineRegistry.new(model_type, backend, device) 决定。

后端	主要优势	使用判断	注意点
FSDP / FSDP2	上手快, Hugging Face 模型支持好	研究、原型、中等规模 dense 模型	MoE 和超大模型扩展性不如 Megatron; 需关注 offload、activation checkpointing。
Megatron / MCore	TP、PP、CP、EP 等并行能力强	百亿到千亿参数、MoE、大集群	新模型适配成本更高, 配置与 checkpoint 转换更复杂。
Automodel / VeOmni / TorchTitan	主线扩展中的统一 engine 方向	特定生态或实验 pipeline	按官方版本与测试矩阵确认成熟度。
NPU / MindSpeed 相关	昇腾硬件支持	Ascend 集群	需按 Ascend tutorial 的模型、算法和精度支持矩阵配置。

Rollout Engine：从批量推理到 server mode

README 中列出的 rollout 后端包括 vLLM、SGLang 和 HF Transformers。安装文档建议 vLLM 0.8.3 及以上更稳定, 并建议打开 `VLLM_USE_V1=1` 以获取更好性能。SGLang 在 multi-turn、agentic RL、VLM RLHF、server-based RL 等方向发展较快。

v0.7 release blog 明确指出, 随着任务从单轮静态生成转向多轮动态交互, 旧式 SPMD rollout 已不足以支撑长尾和工具调用场景, 主线转向 server-based rollout。server mode 下, 推理引擎像在线服务一样接收逐样本请求, 通过动态 batching 提高吞吐; AgentLoop 作为客户端管理工具调用和环境交互。

Checkpoint Engine 与权重同步

强化学习后训练的特殊之处在于 rollout 使用的策略必须不断跟上训练后的 actor。同步训练中, actor 更新后会把最新权重推给 rollout engine。异步或解耦训练中, rollout 和训练可能在不同节点, 权重同步成为独立工程问题。官方主线引入 Checkpoint Engine, 目标是提供统一的权重发送与接收接口, 并支持 NCCL、NIXL 等 transport。

Worker 是角色组合, Engine 是计算实现, Checkpoint Engine 是权重流动。三者分开后, 你才能把算法保持不变, 同时替换 FSDP 为 Megatron, 或把 rollout 从 colocate 改成独立 server。

算法层：PPO、GRPO、DAPO 与优势估计

verl 的算法层不是把每个算法写成互不相干的 trainer，而是尽量复用同一条 rollout、reward、advantage、update 数据流。差异主要落在 advantage estimator、critic 是否启用、KL 放在 reward 还是 loss、采样组大小和 loss aggregation。

PPO: actor-critic 与 GAE

PPO 使用 actor-critic 架构。actor 是被优化的策略，critic 估计 value。生成后，PPO 计算 old policy 与 new policy 的概率比，再用 clipped surrogate objective 限制单次策略更新幅度。GAE 用 `gamma` 和 `lam` 在 bias 与 variance 之间折中。

PPO 配置	作用	起步判断
<code>algorithm.adv_estimator=gae</code>	启用 GAE 优势估计	需要 critic 时使用。
<code>critic.*</code>	critic 模型、优化器、batch 参数	critic 显存可比 actor 小，但仍需规划。
<code>actor.clip_ratio</code>	限制 policy ratio	默认 0.2 是常见起点。
<code>algorithm.use_kl_in_reward</code>	把 KL 作为 reward penalty	PPO/RLHF 常见。
<code>algorithm.kl_ctrl</code>	固定或自适应 KL 系数	reward hacking 或 KL 失控时重点检查。

GRPO: 去掉 critic，用组内相对奖励

GRPO 的核心是对每个 prompt 采样多条回答，使用组内平均奖励作为 baseline。它避免训练 critic，降低显存和计算复杂度，但要求 `rollout.n` 大于 1，并且 reward 必须能区分同一 prompt 下不同回答的质量。

verl 文档提醒，很多配置仍带 `ppo_` 前缀，但可用于 GRPO，因为训练循环与 PPO 近似，只是没有 critic。GRPO 通常设置 `algorithm.adv_estimator=grpo`、`actor_rollout_ref.rollout.n > 1`，并用 `actor.use_kl_loss=True` 在 actor loss 中加 KL。

DAPO: 把 GRPO 式训练推向长 CoT 和大规模复现

DAPO 在 verl 官方算法页中作为 recipe 呈现，并与 DAPO 论文、BytedTsinghua-SIA DAPO 项目关联。它关注长链推理训练中的几个关键问题：动态采样过滤无信息 group、clip higher 调整上下界、token-level loss 改变长回答归一化、overlong reward shaping 控制过长输出。

DAPO 机制	配置入口	解决的问题
Clip Higher	<code>clip_ratio_low</code> , <code>clip_ratio_high</code>	对正负 advantage 使用不对称更新边界, 控制策略更新。
Dynamic Sampling	<code>algorithm.filter_groups.enable</code>	过滤全对或全错 group, 避免无 advantage 信号的样本浪费训练 batch。
Token-level Loss	<code>actor.loss_agg_mode="token-mean"</code>	让长序列训练按 token 聚合, 降低长 CoT 场景中的长度偏置。
Overlong Reward Shaping	<code>reward_model.overlong_buffer</code>	对接近最大长度的输出逐步惩罚, 而不是只做硬截断。

AdvantageEstimator 注册表

当前源码中的 `core_algos.py` 用 `AdvantageEstimator` 枚举和 `registry` 管理优势估计器, 包括 `gae`、`grpo`、`reinforce_plus_plus`、`reinforce_plus_plus_baseline`、`remax`、`rloo`、`opo`、`grpo_passk`、`gpg`、`rloo_vectorized`、`grpo_vectorized`、`optimal_token_baseline`、`tir_optimal_token_baseline`、`gdpo` 等。使用者不必把每个算法都改成新 `trainer`, 很多扩展只需要注册新的 `advantage` 函数或 `policy loss`。

选择算法的实用路线

任务特征	优先算法	理由	额外检查
奖励模型给连续偏好分, 已有 critic 预算	PPO	actor-critic 稳定, KL reward penalty 成熟	critic loss、value clipping、KL controller。
数学/代码可验证奖励, 每题可采多条回答	GRPO	不需要 critic, 组内 baseline 简洁	<code>rollout.n</code> 、group reward 方差、loss aggregation。
长 CoT, 样本常出现全对/全错 group	DAPO	动态采样和 token-level loss 更贴合长推理	overlong reward、clip high、生成 batch 上限。
希望极简或做 baseline	REINFORCE++ / RLOO	实现更简单, 减少 critic 成本	方差、batch size、reward 标准化。

算法层最常见误区

把 `micro_batch_size` 当算法参数调, 或者把 `rollout.n` 当纯性能参数调。前者主要是显存与吞吐, 后者直接改变 GRPO/DAPO 的 group 结构和 advantage 估计。

配置层：Hydra 参数如何改变训练行为

verl 的配置是架构的一部分。它不是简单把命令行参数传给脚本，而是在运行时决定数据、模型、后端、资源、算法、日志、checkpoint 和恢复策略。

配置树的骨架

配置块	控制对象	必须理解的字段
data	数据文件、长度、batch、字段名、截断	train_files , val_files , prompt_key , max_prompt_length , max_response_length , train_batch_size
actor_rollout_ref.model	actor/ref/ rollout 共用模型路径与 HF 配置	path , trust_remote_code , use_remove_padding , enable_gradient_checkpointing
actor_rollout_ref.actor	actor 训练与 policy loss	strategy , ppo_mini_batch_size , ppo_micro_batch_size_per_gpu , use_dynamic_bsz , clip_ratio , use_kl_loss
actor_rollout_ref.rollout	生成后端与采样	name , n , temperature , top_p , tensor_model_parallel_size , gpu_memory_utilization
critic	value model	strategy , model.path , ppo_micro_batch_size_per_gpu
reward / reward_model	奖励模型、奖励函数、reward manager	enable , path , reward_manager , custom_reward_function
algorithm	优势估计与 KL	adv_estimator , gamma , lam , use_kl_in_reward , kl_ctrl , filter_groups
trainer	集群规模、日志、验证、保存、恢复	nnodes , n_gpus_per_node , logger , test_freq , save_freq , resume_mode

batch size 的四个维度

batch 配置是新手最容易误解的地方。verl 官方文档反复强调，含 micro_batch_size 的参数是为了避免 OOM 和调吞吐，理论上不应该改变算法收敛语义；而 train batch、rollout group size 和 mini-batch 会改变训练数据组织。

字段	层级	语义	改变后影响
<code>data.train_batch_size</code>	算法	每次迭代采多少 prompt	影响 on-policy batch、方差、更新频率。
<code>rollout.n</code>	算法	每个 prompt 采多少回答	GRPO/DAPO group 大小，生成成本线性增加。
<code>actor.ppo_mini_batch_size</code>	算法/优化	一次 policy update mini-batch 全局大小	影响梯度估计和优化步数量。
<code>*micro_batch_size_per_gpu</code>	工程	每 GPU 一次 forward/backward 的样本数	主要影响显存、梯度累积、吞吐。
<code>*max_token_len_per_gpu</code>	工程	dynamic batch 的每 GPU token 上限	影响长序列显存和利用率。

最小命令怎么读

```
python3 -m verl.trainer.main_ppo \
  data.train_files=$HOME/data/gsm8k/train.parquet \
  data.val_files=$HOME/data/gsm8k/test.parquet \
  data.train_batch_size=256 \
  data.max_prompt_length=512 \
  data.max_response_length=512 \
  actor_rollout_ref.model.path=Qwen/Qwen2.5-0.5B-Instruct \
  actor_rollout_ref.rollout.name=vllm \
  actor_rollout_ref.rollout.tensor_model_parallel_size=1 \
  actor_rollout_ref.actor.ppo_mini_batch_size=64 \
  actor_rollout_ref.actor.ppo_micro_batch_size_per_gpu=4 \
  critic.model.path=Qwen/Qwen2.5-0.5B-Instruct \
  algorithm.kl_ctrl.kl_coef=0.001 \
  trainer.n_gpus_per_node=1 \
  trainer.nnodes=1
```

这段命令里，数据路径决定样本来源；模型路径决定 actor、critic 的初始权重；rollout 后端决定生成由谁完成；mini/micro batch 决定优化与显存；KL 系数决定策略偏离参考模型的惩罚；节点和 GPU 数决定 Ray 资源池大小。读命令时不要从上到下机械阅读，而要按「数据、模型、生成、算法、资源、日志」分组。

配置审查清单

- 数据字段：prompt_key 是否和 parquet 一致，reward_model.ground_truth 是否存在，uid 是否能支持 group 归因。
- 长度：max_prompt_length 是否误截题目，max_response_length 是否导致大量截断，validation 是否使用不同采样参数。
- 算法：PPO 是否启用 critic，GRPO/DAPO 是否 rollout.n > 1，KL 是放 reward 还是 loss。
- 后端：FSDP2 适合原型，Megatron 适合大规模；rollout TP 要满足模型权重和 KV cache。
- 显存：先调 micro batch、dynamic token limit、gradient checkpointing、activation offload，再调算法 batch。
- 可复现：固定 seed、记录 commit、保存完整 Hydra 配置、保存 reward function 文件和数据预处理脚本。

工程落地：性能、调试、异步、Agentic RL

能跑通 demo 只是开始。真正使用 verl，需要能从日志判断瓶颈、从异常定位层级、从任务形态选择同步或异步 pipeline。

性能瓶颈从哪里看

日志信号	可能瓶颈	优先处理
timing/gen 高	rollout 生成慢、KV cache 不足、TP/DP 不合适、长尾输出	调 gpu_memory_utilization 、 max_num_batched_tokens 、rollout TP/DP、server async。
timing/ref / log-prob 高	forward-only batch 太小或动态 batch 未启用	提高 log_prob_micro_batch_size_per_gpu 或 token limit。
timing/update_actor 高	训练后端吞吐不足，micro batch 太小，通信开销大	gradient checkpointing、dynamic batch、FSDP2、Megatron 并行配置。
KL 快速升高	reward 太强、KL 约束太弱、log-prob 不一致	检查 reference、old_log_probs、reward scale、KL coef。
response_length 接近上限	模型过长输出或被截断	检查 overlong reward、max_response_length、prompt 格式。
reward 全 0 或全 1	打分函数无区分度或数据答案错	抽样 dump generations，逐条验证 reward。

调优顺序

1. 先做正确性。用小模型、小 batch、console logger、少量数据跑 3 到 5 个 step，确认 reward、KL、response length、checkpoint 都正常。
2. 再做显存。OOM 时先降 *micro_batch_size_per_gpu 或 dynamic token limit，打开 gradient checkpointing，再考虑 offload。
3. 再做 rollout 吞吐。调 gpu_memory_utilization 、 max_num_seqs 、 max_num_batched_tokens 、TP/DP；不同推理后端含义不完全相同，不能直接照搬。
4. 最后做算法 batch。只有当系统稳定后，才扩大 train_batch_size 、 rollout.n 或 DAPO dynamic sampling 上限。

Agentic RL 的特殊复杂性

Agentic RL 不只是把 prompt 变长。它引入外部环境等待、工具调用解析、trace、失败恢复、token 与文本不可逆转换等问题。官方 Agentic RL 文档解释了为什么训练期间 server 提供 token-based generate 接口，而不是普通 chat completion API：训练要严格使用推理引擎实际生成的 token，否则 log-prob 和 advantage 会对不上。

工具调用训练通常需要 `data.return_raw_chat=True`、`actor_rollout_ref.rollout.mode=async`，并在数据中加入 `agent_name` 等字段，让 rollout 时选择 `ToolAgentLoop` 或 `SingleTurnAgentLoop`。调试时应启用 `trace` 或 `dump generations`，先看模型是否生成合法工具调用，再看工具结果是否写回上下文，最后看 `reward` 是否按最终轨迹给分。

异步训练的边界

官方 v0.7 口径中，`verl-trainer` 包含同步 `on-policy`、`one-step-off-policy` 和 `fully async` 三类 pipeline。同步最接近算法定义，但会被长尾生成拖慢；一步 `off-policy` 把当前训练和下一批生成重叠，通常在稳定性与吞吐之间折中；全异步把 `trainer` 和 `rollouter` 解耦，适合 128+ GPU 或长链推理等生成严重瓶颈场景，但需要 `staleness control`、权重同步和数据队列能力。

实践中不要过早上全异步。先用同步 pipeline 找到 `reward` 和算法参数，再用日志证明 `timing/gen` 是主要瓶颈，最后再引入异步。否则你会同时调 `reward`、算法、数据传输、权重同步和 `stale policy`，定位成本会陡增。

故障定位树

问题	第一检查点	第二检查点	最终定位层
启动失败	Hydra 配置键是否存在	Ray runtime env 是否安装依赖	配置层 / 环境层
数据报错	parquet 字段和 prompt_key	chat template 与 tokenizer	数据层
reward 异常	dump 5 条输入输出	ground_truth 提取与 reward 函数	任务层
OOM	rollout 还是 actor update	micro batch、KV cache、offload	后端层
训练不涨	reward 是否有方差	KL 是否过强或过弱	算法层
多机 hang	NCCL / Ray 日志	worker world size 与并行配置	控制层 / 后端层

工程判断

`verl` 调试要按层定位。不要看到 OOM 就改算法，不要看到 `reward` 低就调显存，不要看到 Ray 报错就怀疑 ppo。先把现象归到数据、控制、后端、算法或环境其中一层。

附录：源码路线、术语表与来源

这一章给出可执行的源码阅读路线和术语表。读完正文后，按这里的顺序看源码，可以把概念压实到具体文件。

源码阅读路线

顺序	文件 / 目录	你要看什么
1	verl/trainer/main_ppo.py	Hydra 入口、Ray 初始化、TaskRunner、角色映射、dataset 创建。注意该文件已标注将被 main_ppo_sync.py 替代。
2	verl/trainer/main_ppo_sync.py	同步 PPO 新入口、TransferQueue、ReplayBuffer、AgentLoop、RewardLoop、CheckpointEngineManager。
3	verl/trainer/ppo/ray_trainer.py	RayPPOTrainer 的 dataloader、init_workers、fit、reward、advantage、update、validation。
4	verl/protocol.py	DataProto、DataProtoFuture、chunk、concat、union、serialization。
5	verl/single_controller/	WorkerGroup、register decorator、dispatch/collect registry、RayWorkerGroup 绑定。
6	verl/workers/engine_workers.py	TrainingWorker、ActorRolloutRefWorker、init_model、compute_log_prob、update_actor、update_weights。
7	verl/workers/engine/	BaseEngine、FSDP、Megatron、Automodel、VeOmni、TorchTitan 后端实现。
8	verl/workers/rollout/	vLLM/SGLang/HF rollout 适配、server mode、async generate。
9	verl/trainer/ppo/core_algos.py	advantage estimator registry、KL controller、policy loss registry。
10	verl/trainer/config/	Hydra 默认配置，确认字段名和默认值。

扩展路线

新增数据集

写 preprocessing 脚本，把原始样本转成 parquet。确保 prompt、data_source、reward_model.ground_truth 和 extra_info 足够 reward 使用。

新增 reward

实现 compute_score(data_source, solution_str, ground_truth, extra_info) 或配置 custom_reward_function.path/name。先用离线样本单测，不要直接进训练。

新增算法

优先复用 trainer 数据流，注册新的 advantage estimator 或 policy loss。只有控制流顺序变化时，才考虑新 trainer。

新增后端

实现 BaseEngine 子类并注册到 EngineRegistry。worker 层通常无需改动，关键是 forward、train、infer、checkpoint、dispatch 信息。

术语表

术语	定义
Actor	被强化学习优化的策略模型，输出下一个 token 的概率分布。
Critic	估计状态或 token 价值的模型，用于 GAE 等 actor-critic 方法。
Rollout	用策略模型生成回答或轨迹的过程，LLM RL 中通常是最耗时环节。
Reference Policy	KL 约束中的参考模型，常为 SFT 或 base policy。
Reward Model	用模型给回答打分，区别于 rule-based verifier。
RLVR	Reinforcement Learning with Verifiable Rewards，使用可验证规则或环境反馈作为奖励。
HybridFlow	verl 采用的混合控制架构：高层单控制器表达 RL 数据流，底层多进程后端执行重计算。
WorkerGroup	Ray 远端 worker 组在 driver 侧的代理，负责 dispatch、execute、collect。
ResourcePool	Ray 集群中一组 GPU 资源的逻辑划分，用于角色放置。
DataProto	verl 的数据交换协议，包含 TensorDict、non_tensor_batch 和 meta_info。
SPMD	Single Program Multiple Data，多进程执行同一程序，常见于分布式训练后端。
MPMD	Multiple Program Multiple Data，多角色不同程序并行，常用于描述 actor/critic/rollout 等角色组合。
FSDP2	PyTorch Fully Sharded Data Parallel 新版本，在内存和组合性上较 FSDP1 有改进。
Megatron	面向超大模型的并行训练后端，支持 TP、PP、CP、EP 等组合。
Dynamic Batch Size	按 token 数而不是固定样本数组织 micro batch，提高长短序列混合时的利用率。
KL Penalty	惩罚当前策略偏离参考策略，防止 reward hacking 和分布漂移。
Checkpoint Engine	主线架构中的权重保存、加载和训练到 rollout 权重同步抽象。
TransferQueue	主线实验/演进方向，用引用传递和队列降低大数据量跨组件传输成本。

来源

- verl official repository: <https://github.com/verl-project/verl>
- verl official documentation: <https://verl.readthedocs.io/en/latest/>
- HybridFlow paper: <https://arxiv.org/abs/2409.19256>
- DAPO paper: <https://arxiv.org/abs/2503.14476>
- DeepSeekMath / GRPO source paper: <https://arxiv.org/abs/2402.03300>

完整来源、版本口径和事实边界见同目录 sources.md 与 data/source-map.tsv。